

A Zero-Knowledge Proof of Proof for a Verified Node.js Port

Redacted disclosure of a self-hosting certificate layer

Abstract

We describe a *zero-knowledge proof of proof* (zkPoP) layer built on top of an in-progress, machine-checked port of the Node.js core libraries to Lean 4. The port’s own theorems are read off the proof assistant’s environment at build time, digested, and committed to a single 64-bit Merkle root. A certificate consists of an authentication path from a claimed theorem’s leaf to the published root, together with a Fiat–Shamir compiled Schnorr argument of knowledge of an opening of a Pedersen commitment, with the challenge bound to the claimed leaf. Completeness, soundness relative to collision freeness, knowledge soundness, and perfect zero knowledge are all proved inside the same proof assistant, as is the correctness of the executable verifier. The layer is *self-hosting* in two senses: certificates are carried as IPLD over the port’s own DAG-JSON and DAG-CBOR codecs and served through the port’s own HTTP/CGI stack, and the layer is applied to itself — the verifier accepts a certificate whose subject is the theorem asserting that this verifier is correct. We prove that self-application is a strictly ascending ladder rather than a fixed point. This document is deliberately *redacted*: it publishes commitments, sizes, verdicts and the shape of the construction, and withholds statement texts, proof scripts and openings. Every figure quoted here is pinned by a machine-checked theorem in the accompanying artefact, so a reader can check the numbers without being shown the material behind them.

1 Introduction

A verified software port raises an awkward disclosure problem. The claim “this component has been proved correct, with no **sorry** and no bespoke axiom” is only as good as the audit behind it, and the natural way to substantiate it — publishing the sources — is exactly what a rights holder, a competitor-sensitive vendor, or a staged release may not want to do.

The layer described here separates the two. What is published is a single 64-bit commitment to the audited catalogue of proved theorems. What a verifier receives, per query, is a compact certificate: a membership proof against that commitment plus a zero-knowledge argument of knowledge of withheld material bound to the claimed theorem. What is never published is the material itself.

Three things distinguish the construction from a generic Merkle-membership service.

1. **The catalogue is the artefact.** It is not maintained by hand. A command walks the proof assistant’s environment while the commitment module elaborates, and admits exactly the non-internal theorems of the project’s namespaces whose axiom closure lies inside the permitted set — the same criterion the build-time audit enforces. Adding, renaming or removing a theorem changes the root, so the commitment cannot drift away from the artefact.
2. **The verifier’s correctness is itself catalogued.** The executable boolean verifier and the mathematical verification predicate are proved to agree, and that agreement theorem is a member of the catalogue, so it can be certified by the very verifier it describes.

3. **Everything is proved in the same system as the port.** The security statements below are theorems of the artefact, not prose; the limitations are theorems too (§9).

Redaction. §10 states precisely what this document does and does not reveal. In short: shapes, sizes, commitments and verdicts are public; statements, proofs and openings are not.

2 The underlying port

The subject artefact is a port of the pure, well-specified core of the Node.js runtime to Lean 4, developed under a uniform discipline: a total implementation faithful to the upstream behaviour, executable specification tests reproducing the reference outputs of the official documentation, and machine-checked theorems pinning the behaviour down. Covered surfaces include path handling, query strings, URLs, JSON, buffers, assertions, an HTTP/1.1 message layer and request router, a CGI gateway, file and stream reads, and the IPLD stack — the data model, the DAG-JSON and DAG-CBOR codecs, and content identifiers with their multiformats (varint, multihash, base32, base58btc).

Two of these matter for what follows. The IPLD codecs give the certificate a canonical byte encoding, so a digest of a structured value is well defined; and the HTTP/CGI layer gives the verifier a deployment surface without leaving the port.

The build enforces the property the catalogue commits to: an audit pass walks every declaration in the project’s namespaces, collects its axiom closure, and fails the build if anything depends on `sorryAx` or on a non-permitted axiom. At the time of writing it reports *0 sorries* and *0 non-permitted axioms* across the project. The number of declarations it covers is public (§8); their statements are not.

3 The construction

3.1 Records, digests and the catalogue

Each catalogued theorem is a *record* carrying its module, its fully qualified name, the head symbol of its statement, and the pretty-printed statement. A record is mapped into the IPLD data model and digested through the port’s own canonical DAG-CBOR encoding; the digest function is domain-separated for leaves and internal nodes. Writing $D(\cdot)$ for the record digest, the catalogue is the list $R_1, \dots, R_n$ of admitted records, its leaves are $D(R_1), \dots, D(R_n)$, and the published commitment is the Merkle root

$$\text{root} = \mathcal{M}(D(R_1), \dots, D(R_n)).$$

The leaves are proved pairwise distinct, so the leaf of a catalogued theorem identifies it *within* the catalogue — while, as §10 notes, revealing nothing about the statement to an outsider.

3.2 Commitment and argument of knowledge

The hiding layer is a Pedersen commitment in an arbitrary $\mathbb{Z}_q$ -module G with independent generators g, h :

$$\text{com}(m, r) = m \cdot g + r \cdot h.$$

It is proved perfectly hiding, and binding is proved *equivalent* to the hardness of the discrete logarithm: from two openings of one commitment the artefact extracts $\log_g h$ explicitly.

Knowledge of an opening is argued with a Schnorr Σ -protocol, compiled non-interactively by Fiat–Shamir with the challenge derived from the transcript *and* the claimed leaf. Completeness, special soundness with an explicit extractor, and perfect honest-verifier zero knowledge are all

proved; the zero-knowledge statement is strong in the sense that the accepting transcripts are shown to be exactly the witness-free simulator’s outputs, and the honest transcripts are the simulated ones re-indexed by a *bijection* of the prover’s randomness.

3.3 The certificate

Definition 3.1 (Certificate). A certificate for a claimed theorem consists of

root	the published catalogue commitment
leaf	the digest of the claimed theorem’s record
path	the authentication path from leaf to root
com	a Pedersen commitment to withheld material
pf	a Fiat–Shamir Schnorr argument of knowledge of an opening of com

Verification checks the path against the root, and the argument with the challenge bound to **leaf**.

Because the challenge is bound to the leaf, an argument produced for one theorem cannot be replayed for another.

4 What is proved

The following are theorems of the artefact. They are stated here informally; each corresponds to a named declaration (Appendix A).

Theorem 4.1 (Verifier correctness). *The executable boolean verifier that a program runs returns **true** precisely when the mathematical verification predicate holds.*

Theorem 4.2 (Completeness). *For every catalogued theorem, the honest prover produces a certificate that verifies against the published root.*

Theorem 4.3 (Soundness, relative to collision freeness). *Assume the digest is collision free. Then any certificate that verifies against the published catalogue root names a leaf that really is a leaf of the catalogue, and hence corresponds to a genuinely catalogued — and therefore audited, *sorry-free* — theorem.*

Theorem 4.4 (Knowledge soundness). *Two verifying certificates sharing an announcement but with distinct challenges yield, through an explicit extractor, a genuine opening of the commitment.*

Theorem 4.5 (Zero knowledge). *A certificate is accepting exactly when its announcement is the one the witness-free simulator would produce. The accepting set therefore does not depend on the withheld opening, and a certificate reveals nothing about that opening beyond the fact that one exists.*

Theorem 4.6 (Non-malleability of the subject). *Distinct catalogued theorems have distinct leaves, and the challenge is bound to the leaf; a certificate therefore says nothing about theorems it does not name.*

Concretely, the artefact exhibits a certificate for one deliberately declassified theorem of the port — the `node:path` normalisation example from the official documentation — verifies it, shows that tampering with either the claimed leaf or the response makes verification fail, and round-trips the certificate through DAG-CBOR.

5 Certifying the entire proof at once

A batch certificate replaces the single leaf and path with a list of leaf/path pairs, and binds the argument of knowledge to the digest of the whole list. Completeness, soundness, knowledge soundness and zero knowledge are proved for batches, together with the binding property that (under collision freeness) the bound statement determines the list of leaves: the batch cannot be re-cut after the fact.

The artefact instantiates this with a certificate covering *the entire catalogue at once*. It is proved to verify, to be checked against the published root, and — crucially — to name exactly the catalogue: no theorem is left out. Adding a leaf or dropping one is proved to break verification.

6 Self-hosting

6.1 Carried and served by the port itself

The certificate is IPLD data. It travels over the port’s own DAG-JSON and DAG-CBOR codecs and is exposed as a single typed function `zkVerify : map → boolean` in the port’s typed IPLD FFI registry, behind the port’s HTTP router and CGI gateway. The same verdict is proved to be produced over DAG-JSON, over binary DAG-CBOR, and end-to-end through the full HTTP/CGI boundary.

Two service-level properties are worth calling out.

Proposition 6.1 (The published root is enforced at the service). *The service accepts only certificates carrying this project’s audited catalogue root; a certificate against any other root is rejected.*

Proposition 6.2 (The access log is a proof trace). *For every session, replaying each logged request reproduces the logged reply. An auditor therefore re-derives every verdict from the web log alone, with no access to the server.*

6.2 Applied to itself

The catalogue of the base commitment module is read off the environment while that module elaborates, so it cannot contain the certificate layer’s own theorems. Running the catalogue builder a second time, with the whole layer in scope, yields a strictly larger catalogue that provably extends the first and contains the verifier’s correctness theorem along with the certificate’s completeness, soundness, zero-knowledge and batch theorems.

Theorem 6.3 (Self-application). *The executable verifier accepts a certificate whose subject is the theorem stating that this very verifier is correct.*

Tampering with the self-certificate is rejected, and the self-certificate round-trips through DAG-CBOR. A batch certificate covers the whole extended catalogue — the zk layer included — in one object.

Theorem 6.4 (The ladder is strict). *A catalogue can only commit to theorems that already exist, so the theorem asserting that a round’s certificate verifies is never inside the catalogue that certificate commits to. A further round does contain it and certifies it, and in its turn cannot commit to its own verification.*

Self-application therefore climbs a strictly ascending ladder; it does not reach a fixed point. That negative statement is proved, not assumed: it is what prevents the construction from claiming a self-certifying closure it does not have.

7 Usage

A deployment has three roles.

Publisher. Builds the artefact. The build recomputes the catalogue from the environment, re-runs the audit, and emits the root. The publisher releases the redacted bundle of §8 — roots and sizes — and nothing else.

Prover. On request for a theorem it holds, emits a certificate: leaf, authentication path, commitment, argument. One certificate for a single theorem occupies 5 880 bytes as DAG-JSON text and 2 377 bytes as DAG-CBOR, and carries a 210-step authentication path; these figures are pinned by theorem (§8). A batch certificate covering the whole catalogue is one object with one argument of knowledge and one path per theorem.

Verifier. Runs `zkVerify` — as a library call, as an RPC over DAG-JSON or DAG-CBOR, or as a CGI script behind any web server — with the published root compiled in. The answer is `{"ok":true}` or `{"ok":false}`. The verifier needs the root, the certificate, and nothing else: no sources, no statements, no openings. If the verifier keeps its access log, the log alone re-derives its verdicts.

8 Published artefacts and figures

The complete public bundle is the following DAG-JSON document:

```
{"catalogRoot":12282104050709871440,"catalogSize":424,  
  "round2Root":13807856633106066439,"round2Size":540,  
  "selfRoot":7942701633946180972,"selfSize":520}
```

(shown here across three lines; the canonical serialisation is a single line with keys in canonical order.)

Round	Theorems	Commitment
Base catalogue	424	12282104050709871440
Self-applied	520	7942701633946180972
Second round	540	13807856633106066439

Table 1: The three published commitments and their sizes. Each round is proved strictly larger than the previous one.

Other public figures: the audit pass covers 1 613 project declarations (376 in the certificate layer’s namespace) and reports 0 sorries and 0 non-permitted axioms; a single certificate is 5 880 bytes of DAG-JSON, 2 377 bytes of DAG-CBOR, with a 210-step path.

Every number in this section is pinned by a machine-checked theorem in the artefact’s companion module, including the verbatim bundle string above, so the figures can be rechecked without any further disclosure.

9 The honest boundary

Four limitations are stated as theorems in the artefact rather than glossed.

1. **The reference digest is not collision free.** No 64-bit digest can be, and the artefact proves that the concrete collision-freeness hypothesis used by the soundness theorems is false for the reference primitive. Every soundness statement therefore carries collision freeness as an *explicit* hypothesis. A deployment instantiates the digest with a wide hash — the SHA-256 behind a real multihash/CID — and inherits the assumption from that primitive.
2. **A certificate is only as good as the root it is checked against.** For *any* digest whatsoever, a verifying certificate against a self-chosen one-leaf root exists; this is proved. Verification is meaningful only against the published root, which is why the service refuses foreign roots.
3. **The layer transports soundness; it does not create it.** What makes the catalogued statements true is that the proof assistant checked their proofs and the build-time audit found no `sorry` and no bespoke axiom. The certificate transports that fact compactly and privately. It does not substitute for it, and a reader who distrusts the audit should distrust the certificate.
4. **The reference group is a demonstration instantiation.** The concrete demo uses a small prime field acting on itself, where discrete logarithms are easy and binding is therefore vacuous *there*; it exercises the data path end to end. Every general theorem is stated for an arbitrary $\mathbb{Z}_q$ -module, so a secure deployment is a matter of supplying a group in which discrete log is hard.

10 Disclosure policy

This document is written to be publishable without releasing the artefact.

Published. The construction and its security statements; the three commitments and catalogue sizes; aggregate audit counts; wire sizes and verdicts; the shape of the certificate and of the service interface; one deliberately declassified example theorem, taken from public Node.js documentation.

Withheld. Statement texts and names of catalogued theorems; proof scripts; module inventories beyond aggregate counts; the withheld openings and their commitments’ preimages; the concrete generator choices and parameters of a production instantiation.

Why the withholding is not merely editorial. Two of the withheld categories are protected by the construction itself rather than by omission.

Proposition 10.1 (The argument reveals nothing about the opening). *Accepting certificates are exactly the simulator’s outputs; the accepting set is independent of the witness.*

Proposition 10.2 (A published leaf does not determine a statement). *There exist distinct records with the same digest. A leaf therefore cannot be inverted to a statement: publishing leaves is not publishing statements.*

The second is proved in the artefact. It is, of course, an information-theoretic statement about the digest and not a claim of preimage resistance: an adversary who can guess a statement can confirm the guess by digesting it. Redaction of statement texts is what prevents *enumeration*; the collision statement is what prevents *inversion*. Neither replaces the other, and a deployment that regards its theorem names as sensitive should additionally salt the records — the commitment layer already carries a hiding component suited to that.

11 Related constructions and discussion

The layer is a conventional composition — Merkle commitment, Pedersen commitment, Schnorr Σ -protocol, Fiat–Shamir — assembled inside a proof assistant and applied to that assistant’s own output. The interest is not in the cryptographic novelty of the components but in three properties of the assembly: the committed set is *derived from the environment* rather than curated, so it cannot silently disagree with the artefact; the verifier’s correctness is a member of the committed set, so the layer can be turned on itself; and the negative results — the non-collision-freeness of the reference digest, the forgeability against self-chosen roots, the strictness of the self-application ladder — are proved alongside the positive ones, so the trust boundary is machine-checked rather than asserted.

The obvious next steps are a wide-hash instantiation (making the collision hypothesis credible rather than provably false), a group with hard discrete logarithm, and salting of records for deployments where theorem names are themselves sensitive.

A Index of named results

For readers with access to the artefact, the results above correspond to the following declarations. The list is given for cross-reference only; the statements themselves are not reproduced here.

Result	Declaration
Verifier correctness	<code>Certificate.verifyBool_iff</code>
Completeness	<code>pop_complete</code>
Soundness	<code>pop_membership</code> , <code>pop_catalog_membership</code>
Knowledge soundness	<code>pop_knowledge</code>
Zero knowledge	<code>pop_zero_knowledge</code> , <code>pop_accepting_witness_free</code>
Subject binding	<code>pop_leaf_determines_record</code>
Batch layer	<code>batch_complete</code> , <code>batch_membership</code> , <code>batch_knowledge</code> , <code>batch_zero_knowledge</code> , <code>batch_stmt_determines_leaves</code>
Entire catalogue	<code>entire_catalog_certificate_verifies</code> , <code>_spec</code> , <code>_covers</code>
Self-application	<code>Self.self_certificate_verifies</code> , <code>Self.self_batch_certificate_verifies</code>
Strict ladder	<code>Self.self_verification_not_in_self_catalog</code> , <code>Round2.round2_certificate_verifies</code> , <code>Round2.round2_verification_not_in_round2_catalog</code>
Service	<code>zk_demo_json</code> , <code>zk_demo_cbor</code> , <code>zk_demo_gateway</code> , <code>zk_foreign_root_rejected</code> , <code>zk_log_is_proof_trace</code>
Honest boundary	<code>digest_not_injective</code> , <code>Merkle.not_collisionFree</code> , <code>pop_root_forgeable</code>
Published figures	<code>Paper.paper_catalog_size</code> , <code>Paper.paper_self_catalog_size</code> , <code>Paper.paper_round2_size</code> , <code>Paper.paper_ladder_strict</code> , <code>Paper.paper_public_bundle_json</code> , <code>Paper.paper_certificate_wire_sizes</code> , <code>Paper.paper_bundle_verdict</code> , <code>Paper.paper_leaf_is_lossy</code>